DIPLOMA IN CYBERSECURITY

UNIT 9

Cross-Site Request Forgery (CSRF)

Attack Demonstration | Anti-CSRF Tokens | Defence in Depth

Student Handout

WARNING: Legal and ethical reminder

The attack in this unit must only be run against the MiniBank lab application supplied with these notes, on your own computer. Sending forged requests to a real website or to another person's account is a criminal offence under the Computer Misuse and Cybercrimes Act, 2018 (Kenya) and similar laws elsewhere.

Lesson Overview and Timing Plan

Learning outcomes

By the end of this session, learners should be able to:

1. Define CSRF and explain why browsers make it possible (automatic cookie attachment).
2. Distinguish same-origin from same-site, and explain what the same-origin policy does and does not prevent.
3. State the three conditions that must hold for a CSRF attack to work.
4. Describe GET-based, POST-based, JSON/API-based and login CSRF variants.
5. Carry out a CSRF attack against a deliberately vulnerable lab application and explain each step in terms of HTTP requests.
6. Explain how anti-CSRF tokens (synchroniser token pattern) work and why an attacker cannot obtain or guess them.
7. Implement anti-CSRF protection in a web application, combining tokens, SameSite cookies and Origin checks.
8. Compare CSRF defences (tokens, double-submit cookies, SameSite, Origin/Fetch Metadata, re-authentication) and identify common mistakes.
9. Test an application for CSRF and verify that a fix works.

Prerequisites and lab requirements

- **Prior knowledge:** HTTP requests and responses, cookies and sessions, HTML forms (Unit 6), basic Python reading skills.
- **Software:** Python 3.9+ with Flask (`pip install flask`), a text editor, curl, and **Firefox** (recommended for the POST demo; see the browser note in Section 4).
- **Lab files (supplied with these notes and listed in Appendix A):** `vulnerable_app.py`, `secure_app.py`, `evil_post.html`, `evil_get.html`, `test_csrf.py`.
- **Network:** everything runs on the learner's own machine (127.0.0.1). No internet or shared network is needed.

Quick setup and self-test

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install flask
python test_csrf.py              # self-test: should end with ALL TESTS PASSED
```

1. Introduction to CSRF

1.1 What is CSRF?

Cross-Site Request Forgery (CSRF), also written XSRF and sometimes called “session riding” or a “one-click attack”, is an attack in which a malicious website, e-mail or message causes a victim’s **browser** to send an unwanted, state-changing request to another site where the victim is **already logged in**. The target site receives a request that carries the victim’s valid session cookie and therefore believes it is a genuine action by the victim.

> **KEY POINT: The core idea**
> The attacker never steals the victim’s password or session cookie. The attacker **borrows the victim’s browser**. Because browsers automatically attach cookies to every request for a site, the forged request arrives with the victim’s credentials already attached.

Analogy: you work in an office and wear an ID badge that opens every door automatically. A stranger slips a note into your hand saying “please press the button to release the safe”, and you press it without reading carefully. The safe opens because *your badge* was recognised. The stranger never had a badge; they only tricked you into using yours.

1.2 Where CSRF sits in the OWASP picture

- CSRF appeared as its own category in older OWASP Top 10 editions (for example 2013 and 2017). It was dropped from the dedicated list after 2017 because many frameworks now protect against it by default.
- Weakness **CWE-352 (Cross-Site Request Forgery)** is grouped under **Broken Access Control** in recent OWASP editions, because the application fails to verify that a request truly came from the user’s own intentional action. Learners should check the current mapping at owasp.org when preparing assignments.
- In the tier model from Unit 6, the weakness lives in the **application tier** (the server accepts the request without verifying intent), the attack is *delivered through* the **client tier** (the victim’s browser), and the damage is done to the **data tier** (records changed).
- CSRF is still found in custom code, legacy applications, APIs that use cookie authentication, and IoT or router web interfaces.

1.3 What CSRF can and cannot do

CSRF CAN	CSRF CANNOT
Make a victim perform actions: transfer money, change e-mail or password, add an administrator, delete data, post content, change settings	Read the response, because the same-origin policy stops the attacker’s page from reading what the target site returns
Work against any action the victim is allowed to do	Act as a user who is **not** logged in (there is no cookie to ride)
Be delivered by a link, an image, a hidden form, an e-mail or a forum post	Bypass a correctly implemented unpredictable token

CSRF CAN	CSRF CANNOT
Be chained with other attacks (change e-mail, then reset password, then take over the account)	Directly steal data (but it can change an e-mail address to one the attacker controls and then use a password reset)

1.4 Why learn CSRF?

- It is a classic example of a flaw caused by **how the web is designed**, not by a typo in code.
- It teaches the difference between **authentication** (who you are) and **intent** (did you really mean to do this?).
- The fix is small, concrete and testable, which makes it ideal for practising secure coding.
- It appears in exams, certifications (Security+, CEH, OSCP-style courses) and in real penetration test reports.

2. Foundations: What Makes CSRF Possible

2.1 Cookies and sessions: a quick recap

HTTP is stateless, so after login a server issues a **session identifier** and the browser stores it in a **cookie**. For every later request to that site, the browser **automatically** adds the cookie. This is convenient, and it is exactly the behaviour that CSRF exploits: the browser does not ask *why* the request is being made or *which page* started it.

Figure 2.1: The server sees only a valid cookie, not who or what triggered the request

```
POST /transfer HTTP/1.1
Host: localhost:5000
Cookie: session=eyJ1c2VyIjoiYWxpY2UifQ.Z...        <-- added AUTOMATICALLY by the browser
Content-Type: application/x-www-form-urlencoded

to=mallory&amount=500
```

2.2 Same-origin policy: what it blocks and what it does not

Browsers enforce the **same-origin policy (SOP)**. An **origin** is the combination of scheme, host and port. The SOP stops a script on one origin from **reading** responses from another origin.

> **WARNING: The crucial misunderstanding**
> The SOP restricts **reading** cross-origin responses. It does **not** stop a page from **sending** certain cross-origin requests, such as submitting an HTML form or loading an image or link. That is why CSRF works: the attacker does not need to read anything; they only need the request to be sent.

2.3 Origin vs site

Two related ideas are often confused:

- **Origin** = scheme + host + port. `http://localhost:5000` and `http://localhost:8000` are **different origins**.
- **Site** = the registrable domain (for example `example.co.ke`), ignoring sub-domains and ports. `https://app.example.co.ke` and `https://api.example.co.ke` are the **same site**.
- Cookie rules such as **SameSite** work on **sites**, not origins. This difference matters in the lab (see below).

Request from page …	To …	Same origin?	Same site?
https://app.example.com/a	https://app.example.com/b	Yes	Yes
https://app.example.com	https://app.example.com:8443	No (port)	Yes
https://app.example.com	https://api.example.com	No (host)	Yes
https://example.org	https://app.example.com	No	**No (cross-site)**
http://127.0.0.1:8000 (lab attacker)	http://localhost:5000 (lab bank)	No	**No (cross-site)**
http://localhost:8000	http://localhost:5000	No (port)	**Yes: same-site!**

> **NOTE: Lab pitfall**
> If the attacker page is served from `localhost:8000` and the bank from `localhost:5000`, the browser treats them as the **same site** (ports are ignored), so SameSite cookies will be sent and the demo will not show what a real cross-site attack looks like. That is why the lab uses `127.0.0.1` for the attacker and `localhost` for the bank.

2.4 Safe and unsafe HTTP methods

Method	Intended use	Should it change state?	CSRF relevance
GET	Retrieve	**No** (safe)	If an application changes data on GET, any link, image or redirect can trigger it
HEAD, OPTIONS	Metadata	No	Not a CSRF target
POST	Submit data	Yes	HTML forms can send cross-site POSTs without any script
PUT, PATCH, DELETE	Modify / remove	Yes	Cannot be sent by plain HTML forms; need JavaScript and are usually subject to CORS preflight, but sometimes tunnelled through POST

Rule: **never change state on GET.** Even so, using POST alone does *not* prevent CSRF, because forms can send POST requests cross-site.

2.5 Cross-site requests a browser will happily send

Technique	Example	Method	Cookies attached?
Link / navigation	`<a href="https://bank/transfer?...">` or `window.location`	GET	Yes (unless SameSite=Strict)
Image or other subresource	`<img src="https://bank/transfer?...">`	GET	Depends on SameSite (not sent for Lax/Strict cross-site)
Auto-submitted form	`<form method="POST" action="https://bank/transfer">`	POST	Depends on SameSite (not sent for Lax/Strict cross-site)
JavaScript fetch/XHR	`fetch(url, {credentials:"include"})`	Any	Only if CORS allows credentials; non-simple requests trigger a preflight

3. The Three Conditions for a CSRF Attack

A CSRF attack succeeds only when **all three** conditions below hold. Teaching this checklist helps learners both find and fix the flaw.

1. **A relevant action exists.** The target application lets a logged-in user do something worth forcing: transfer money, change an e-mail address, add an admin, delete an account, post a message, change a vote.
2. **The application relies only on information the browser sends automatically** (a session cookie, HTTP Basic credentials, or a client-TLS certificate) to authenticate the request, with no extra value the attacker cannot see or guess.
3. **The request has no unpredictable parameter the attacker cannot know**, such as a correct anti-CSRF token. If every parameter in the request (to, amount, action) is something the attacker can set or guess, the request can be forged in full.

> **KEY POINT: In one sentence**
> If an attacker can predict the entire request **except** the part the browser adds automatically (the cookie), and the server accepts it anyway, CSRF is possible.

3.1 The MiniBank lab application

For the rest of this unit we use **MiniBank**, a two-file Flask demo supplied with these notes (Appendix A). It has two users, `alice` (balance 1000) and `mallory` (balance 50), and one meaningful action: **transfer money**.

Figure 3.1: vulnerable_app.py (shortened) — checks against the three conditions below

```
@app.route("/transfer", methods=["GET", "POST"])     # VULNERABLE VERSION
def transfer():
    user = session.get("user")
    if not user:
        return redirect("/login")
    to = request.values.get("to", "")
    amount = int(request.values.get("amount", "0"))
    USERS[user]["balance"] -= amount
    USERS[to]["balance"] += amount
    return redirect("/?msg=Sent")
```

Condition	Does MiniBank meet it?
1. A relevant action exists	Yes: `/transfer` moves money between accounts
2. Authentication is only the automatic session cookie	Yes: the route trusts `session["user"]`, set purely from the cookie
3. No unpredictable value is required	Yes: `to` and `amount` are ordinary parameters; the attacker can set both and there is no token to guess

All three conditions hold, so MiniBank is exploitable, and the route even accepts GET as well as POST, which will matter in Demo 2.

4. Demonstration: Carrying Out a CSRF Attack

This section is a script for a live, narrated demonstration. Run every command yourself before class and note the actual output you see, because small version differences can change exact wording.

4.1 Set the stage

1. Start the vulnerable server: `python vulnerable_app.py` (serves on `http://localhost:5000`).
2. In **Browser Window 1**, go to `http://localhost:5000`, log in as `alice` / `alice123`. Show the home page: "Balance: KES 1000".
3. Open DevTools (F12) → Application/Storage → Cookies, and show the `session` cookie. Point out that it has **no SameSite value shown** (treated according to the browser's default) and ask: "What happens if this cookie is sent somewhere alice did not intend?"
4. Explain the scene: Mallory wants Alice's money and has prepared a web page at `http://127.0.0.1:8000` (a different **site**, simulating "anywhere on the internet").

4.2 Demo 1: GET-based CSRF (the simplest case)

Because the vulnerable `/transfer` route also accepts GET, the simplest possible attack is a single link or an auto-redirecting page.

Figure 4.1: evil_get.html — a top-level navigation to the vulnerable endpoint

```
<!-- evil_get.html -->
<!doctype html><html><body>
  <h1>Loading your deal...</h1>
  <script>
    window.location = "http://localhost:5000/transfer?to=mallory&amount=300";
  </script>
</body></html>
```

1. Serve the attacker page: `python -m http.server 8000` in the folder containing `evil_get.html`.
2. In **Browser Window 1** (still logged in as Alice), open `http://127.0.0.1:8000/evil_get.html`.
3. The browser silently navigates to the bank, the cookie is attached, and the transfer executes.
4. Switch back to the MiniBank tab and refresh: **"Balance: KES 700"** (1000 − 300). Alice never clicked "Send".

> NOTE: **Why this works even with modern defaults**
> A top-level **navigation** (changing `window.location`, or a plain `<a>` link) is sent even when the cookie is `SameSite=Lax`, because Lax specifically allows cookies on top-level GET navigations. Only `SameSite=Strict` would block it. This is an important, often-missed detail: **Lax is not enough to stop every GET-based CSRF.**

4.3 Demo 2: POST-based CSRF (the classic case)

Most real actions use POST, and the classic CSRF payload is a hidden auto-submitting form.

Figure 4.2: evil_post.html — a disguised page that auto-submits a transfer

```
<!-- evil_post.html -->
<!doctype html><html>
<body onload="document.getElementById('f').submit()">
  <h1>Congratulations! Claim your free airtime voucher...</h1>
  <form id="f" action="http://localhost:5000/transfer" method="POST">
    <input type="hidden" name="to" value="mallory">
    <input type="hidden" name="amount" value="500">
  </form>
</body></html>
```

1. Log in as Alice again (balance reset by restarting the app, or continue from 700).
2. Open `http://127.0.0.1:8000/evil_post.html`. The page shows an innocent-looking message; the form submits itself invisibly.
3. Refresh the MiniBank tab: the balance has dropped by 500. Open DevTools → Network on a repeat visit to show the POST request, its destination, and that **no password or token was needed**, only the session cookie the browser attached automatically.

> **WARNING: A note on today's browser defaults**
>
> Chrome, Edge and Opera now treat a cookie with **no SameSite attribute as `Lax` by default**, and Lax blocks cookies on cross-site POST requests, so this exact demo may **not** work unmodified in those browsers unless the cookie is explicitly set to `SameSite=None`. Firefox and Safari have historically not applied this default in the same way. **Test in your demo browser beforehand** (Section "Before class"). To guarantee the demo works in any browser for teaching purposes, you may temporarily run the vulnerable app with `COOKIE_SAMESITE=None COOKIE_SECURE=1` over HTTPS, or simply demonstrate Demo 1 (GET) live and walk through Demo 2 as a narrated code-and-request walkthrough. This browser behaviour is itself an excellent teaching point: it shows that **SameSite is a real defence**, covered in Section 6.

4.4 Demo 3: Proving it with curl (what the browser does, made explicit)

curl does not run JavaScript or enforce SameSite, so it is a clean way to show exactly what request the browser assembles and sends.

Verified output from the supplied vulnerable_app.py

```
# 1. Log in and save the session cookie to a cookie jar
$ curl -s -c jar.txt -d "username=alice&password=alice123" http://localhost:5000/login

# 2. Check the balance
$ curl -s -b jar.txt http://localhost:5000/ | grep Balance
Balance: KES 1000

# 3. Forge a transfer "from another site" (simulated with a fake Origin header)
$ curl -s -b jar.txt -H "Origin: http://127.0.0.1:8000" \
       -d "to=mallory&amount=500" http://localhost:5000/transfer

# 4. Check the balance again
$ curl -s -b jar.txt http://localhost:5000/ | grep Balance
Balance: KES 500
```

Discussion: the only thing that proved "who" sent the request was the cookie, and the cookie was supplied from a saved jar, exactly as a browser would attach it automatically. The forged `Origin` header was not even checked, because the vulnerable application performs **no verification of intent at all**.

4.5 What the three demos proved

- A victim who is merely **logged in and browsing** (not doing anything wrong) can have state changed on another site without clicking any “Confirm” button on that site.
- Neither HTTPS nor a strong password protects against this: the session was already legitimately established.
- The vulnerability is **not in the browser** and not in the user’s behaviour; it is in the **server trusting a request based only on a cookie**.

5. Variants, Impact and Related Attacks

5.1 Variants of CSRF

Variant	Description	Example
GET-based	State change triggered by a simple link, image tag or redirect	Demo 1 above; `<img src="https://site/delete?id=1">`
POST-based	Auto-submitting hidden form	Demo 2 above
JSON / API CSRF	An API expects JSON and relies only on cookies for auth; attacker uses a form with `enctype` tricks or a simple content type to approximate JSON, or exploits an API that accepts form-encoded bodies	A mobile-style API reachable from the browser with cookie auth and no token check
Login CSRF	The attacker forges a login request using the **attacker's own account**, logging the victim into the attacker's session, so anything the victim later saves (for example a search history or a stored card) lands in an account the attacker controls	Victim is silently logged into "attacker's" shopping account and enters their card details there
Multi-step / chained CSRF	Several forged requests combine to defeat multi-step protections, for example change e-mail then request a password reset	Change e-mail (CSRF) → trigger "forgot password" → attacker receives the reset link

5.2 CSRF vs related attacks: do not confuse them

Attack	What happens	Relationship to CSRF
CSRF	Victim's browser is tricked into sending an unwanted authenticated request to a site where they are logged in	This unit
XSS (cross-site scripting)	Attacker's script runs **inside** the target site's own page in the victim's browser	Much more powerful: can read responses, steal tokens directly, and **defeat many CSRF defences** because it runs same-origin
Clickjacking	The target page is invisibly framed and the victim is tricked into clicking a real button on it	A different delivery technique; also defeated by frame-busting / `X-Frame-Options`
SSRF (server-side request forgery)	The **server itself** is tricked into making a request to an attacker-chosen destination	Different: in CSRF the browser is the unwilling messenger; in SSRF the server is
Session hijacking	Attacker obtains and reuses the victim's actual session identifier	CSRF never needs to see or steal the cookie at all

> **WARNING: Important relationship: XSS breaks most CSRF defences**
>
> If an attacker can run JavaScript on the **same origin** as the target application (via XSS), that script can read the page's anti-CSRF token and include it in a forged request, defeating standard CSRF protection entirely.

This is why OWASP treats **XSS prevention as a prerequisite for CSRF defences to be meaningful**, and it is worth stating plainly to learners: *CSRF tokens protect against a different, weaker attacker than XSS does.*

5.3 Real-world impact

REAL-LIFE SCENARIO: **Illustrative historical cases**

Home routers (recurring issue): many consumer router web interfaces have been found vulnerable to CSRF in their admin pages, allowing a malicious web page visited by an administrator to silently change DNS settings or Wi-Fi passwords, since the admin session cookie (or even default credentials cached by the browser) was trusted without a token.

Social media "like" and "follow" forgery: CSRF has historically been used to make a logged-in visitor silently "like", "follow" or post on a user's behalf by loading a hidden image or form from an attacker-controlled page.

Generic lesson: CSRF rarely makes headlines on its own, but it is consistently found in penetration tests of custom-built admin panels, banking demo systems, and IoT device interfaces, precisely the kind of application diploma graduates are likely to build and test.

Business impact of a successful CSRF attack can include unauthorised financial transactions, account takeover (through e-mail or password changes), data loss (deleted records), reputational damage and regulatory exposure under data-protection law.

6. Defending Against CSRF

Good CSRF defence usually combines **more than one** layer. We cover the synchroniser token pattern in depth (it is what we implement in Section 7), then the supporting defences.

6.1 Defence 1: Anti-CSRF tokens (synchroniser token pattern)

The server generates a **long, random, unpredictable token** and stores it (commonly in the user's session). The token is embedded in every form (or exposed to JavaScript for API calls). On a state-changing request, the server checks that a token was submitted **and** that it matches the one on record for that session. An attacker's page cannot read the victim's page (same-origin policy) and so cannot discover the correct token, and cannot guess a long random value.

Figure 6.1: The synchroniser token pattern

```
Step 1  Server creates a random token and stores it in the session.
Step 2  Server embeds the token in a hidden field of every form that changes state:
          <input type="hidden" name="csrf_token" value="Xk93f...">
Step 3  Browser submits the form; the token travels as ORDINARY request data,
        NOT as a cookie, so it is not attached automatically.
Step 4  Server compares the submitted token with the one stored in the session.
        Match  -> process the request.
        No match / missing -> reject (HTTP 403), fail CLOSED.
```

Requirement	Why it matters
Unpredictable (cryptographically random, long enough — 128+ bits)	Must not be guessable, sequential or derived from public data
Tied to the session (or otherwise bound per-user)	A token stolen or guessed for one user must not work for another
Checked on every state-changing request, using a constant-time comparison	A single unchecked route reopens the hole; timing attacks can leak tokens if compared insecurely
Not itself sent automatically by the browser	Must travel as form data, a custom header, or JSON body — never purely as a cookie
Regenerated at login (and optionally per sensitive action)	Prevents session-fixation-style reuse of a token known before authentication

Token delivery methods

Method	How	Typical use
Hidden form field	`<input type="hidden" name="csrf_token" value="...">`	Traditional server-rendered HTML forms
Custom request header	`X-CSRF-Token: <token>`, read from a cookie or meta tag by JavaScript and attached to `fetch`/XHR calls	Single-page applications calling a JSON API
Double-submit cookie	Token stored in both a (non-HttpOnly) cookie and sent back as a header/field; server checks the two match	Stateless APIs where storing a per-session token server-side is inconvenient

> **NOTE: Double-submit cookie: use the signed/HMAC variant**
> The naive double-submit pattern (just check the cookie value equals the submitted value) can be weakened if an attacker can set cookies on a related sub-domain. The more robust variant has the server **sign** the token (HMAC with a secret key) so that only the server could have produced a valid pairing.

6.2 Defence 2: SameSite cookies

The `SameSite` cookie attribute tells the browser when to attach a cookie to a request that originates from another site.

Value	Behaviour	CSRF protection
`None`	Cookie sent on every request, including cross-site (requires `Secure`)	No protection
`Lax`	Cookie sent on same-site requests and on **top-level GET navigations**, but not on cross-site POST, image loads, iframes, etc.	Blocks most POST-based CSRF; does **not** block GET-based CSRF via navigation (see Demo 1)
`Strict`	Cookie sent only on same-site requests, even top-level navigation from an external link	Strongest, but can break legitimate flows (for example arriving at a logged-in page by clicking a link from an e-mail)

As of the current generation of major browsers, cookies that do not explicitly set `SameSite` are commonly treated as `Lax` by default in Chrome, Edge and Opera; Firefox and Safari have different histories and defaults, and all of this continues to evolve. **Always set `SameSite` explicitly** rather than relying on a browser default, and verify current behaviour for your target browsers before teaching or deploying.

> **WARNING: SameSite is a browser-side mitigation, not a server-side fix**
> SameSite depends entirely on the visitor's browser correctly implementing and applying it. Old or unusual browsers, misconfigured proxies, or certain app embeddings (webviews) may not apply it. SameSite should be a **layer**, not the only defence, which is why OWASP still recommends tokens as the primary control.

6.3 Defence 3: Verifying Origin or Referer

On a state-changing request, the server can check the `Origin` header (present on most cross-origin POSTs and many same-origin ones) or, if absent, the `Referer` header, and reject the request if it does not match the application's own origin.

- **Strength:** simple, effective against classic CSRF, and recommended by OWASP as a strong **defence-in-depth** layer alongside tokens.
- **Weaknesses:** `Referer` can be stripped by privacy settings or proxies; some legacy or mobile clients omit `Origin`. Never rely on this as the **only** control; prefer "reject if present and wrong, but do not rely solely on it being present."
- A newer, more robust relative is the **`Sec-Fetch-Site` Fetch Metadata header**, sent automatically by modern browsers, which explicitly states whether a request is `same-origin`, `same-site`, `cross-site` or `none`.

6.4 Defence 4: Re-authentication and step-up checks

For the most sensitive actions (changing a password, adding a payment recipient, large transfers), ask the user to re-enter their password, a one-time code, or confirm through a second channel. This defeats CSRF (and several other attack types) because the attacker cannot forge something only the real user can supply.

6.5 Defences that do NOT work

Common mistake	Why it fails
Using POST instead of GET, with nothing else	Forms can submit POST requests cross-site too (Demo 2)
Checking only that a cookie is present	The attacker never needed the cookie's value; the browser attaches it for them
Putting the token only in a **GET** URL (query string)	Leaks through browser history, referer headers and server logs, and a GET request can still be auto-triggered
A token that is not tied to the session (same token for everyone, or never changes)	A token leaked once (for example through logs or a shared screenshot) is a skeleton key
Relying on `X-Requested-With` headers alone	Easy to add to forged requests when the attacker uses script; not a secret
CAPTCHA on every form	Hurts usability and does not verify *origin* of the request, only that *a* human solved a puzzle at *some* point

6.6 A layered recommendation

1. Use a well-tested framework feature for anti-CSRF tokens wherever one exists; do not hand-roll cryptography.
2. Set `SameSite=Lax` (or `Strict` where usable) on session cookies, explicitly.
3. Verify `Origin` (falling back to `Referer`) on state-changing requests as an extra layer.
4. Require re-authentication for high-value actions.
5. Prevent XSS (Unit 6, A05) — without this, token-based defences can be bypassed.
6. Test regularly (Section 8).

7. Implementing Anti-CSRF Tokens (Live Coding)

We now fix MiniBank. The secure version is supplied as `secure_app.py` (Appendix A); this section walks through it piece by piece so it can be live-coded or read as a guided build. All code below was run and verified (Section 7.6).

7.1 Step 1: Harden the session cookie

```
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,     # JavaScript cannot read the cookie (helps against XSS
token theft)
    SESSION_COOKIE_SAMESITE="Lax",    # set explicitly -- never rely on the browser default
    SESSION_COOKIE_SECURE=True,       # cookie only sent over HTTPS (set False only for local
http lab testing)
)
```

7.2 Step 2: Generate a per-session token

```
import secrets

def get_csrf_token():
    """Create a random token bound to this session (once) and return it."""
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_urlsafe(32)   # 256 bits of randomness
    return session["csrf_token"]


app.jinja_env.globals["csrf_token"] = get_csrf_token          # usable in templates as {{
csrf_token() }}
```

`secrets.token_urlsafe` uses a cryptographically secure random generator, unlike `random`, which must never be used for security tokens because its output can be predicted.

7.3 Step 3: Embed the token in every state-changing form

```
<form method="POST" action="/transfer">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  To: <input name="to" value="mallory">
  Amount: <input name="amount" value="10">
  <button>Send</button>
</form>
```

7.4 Step 4: Verify the token (and Origin) before any state change

Figure 7.1: central check applied to every request (Section 4.4, Unit 4 and Unit 6's "fail closed" principle)

```
import hmac
from flask import request, abort


@app.before_request
def csrf_protect():
    if request.method in ("GET", "HEAD", "OPTIONS"):
        return                                              # safe methods never change state
    # Extra layer: reject if the browser says this is cross-origin
    origin = request.headers.get("Origin")
    if origin and origin != request.host_url.rstrip("/"):
        abort(403, "Cross-origin request blocked")
```

```
    # Core defence: token must be present AND match the session, compared safely
    sent = request.form.get("csrf_token") or request.headers.get("X-CSRF-Token", "")
    expected = session.get("csrf_token", "")
    if not expected or not hmac.compare_digest(sent.encode(), expected.encode()):
        abort(403, "Invalid or missing CSRF token")        # FAIL CLOSED
```

- **`@app.before_request`** centralises the check (Unit 6, Section 7.1's advice: enforce access/intent control in one place, on every request) instead of repeating it in every route, where it is easy to forget one.
- **`hmac.compare_digest`** performs a constant-time comparison, so the check does not leak how many leading characters matched through response-time differences.
- The check **fails closed**: any problem (missing token, wrong token, wrong origin) results in a 403, never in silently allowing the request.

7.5 Step 5: Make the route itself safer

```
@app.route("/transfer", methods=["POST"])   # GET removed: state changes are POST-only now
def transfer():
    user = session.get("user")
    if not user:
        return redirect("/login")
    to = request.form.get("to", "")
    amount = int(request.form.get("amount", "0") or 0)
    if to not in USERS or amount <= 0 or amount > USERS[user]["balance"]:
        return redirect("/?msg=Transfer+refused")
    USERS[user]["balance"] -= amount
    USERS[to]["balance"] += amount
    return redirect("/?msg=Sent")
```

Also regenerate the token at login (`session.clear()` before setting the new user) so a token observed or guessed before authentication cannot be reused afterwards, and protect `/logout` with a token too, since forcing an unwanted logout can be used to prepare further attacks (for example a login-CSRF).

7.6 Verifying the fix

Re-running the same attacks from Section 4 against `secure_app.py` gives the following, verified results:

Actual output captured from secure_app.py during preparation of this unit

```
$ curl -s -c jar.txt -d "username=alice&password=alice123&csrf_token=<pre-login token>" \
       http://localhost:5000/login

# Forged POST with NO token:
$ curl -s -b jar.txt -d "to=mallory&amount=500" http://localhost:5000/transfer -o /dev/null -w
"%{http_code}\n"
403

# Forged POST with a WRONG token:
$ curl -s -b jar.txt -d "to=mallory&amount=500&csrf_token=wrong" http://localhost:5000/transfer
-w "%{http_code}\n" -o /dev/null
403

# Forged POST with the RIGHT token but a foreign Origin header (simulated attacker site):
$ curl -s -b jar.txt -H "Origin: http://127.0.0.1:8000" -d
"to=mallory&amount=500&csrf_token=<correct>" \
       http://localhost:5000/transfer -o /dev/null -w "%{http_code}\n"
403
```

```
# Legitimate request: correct token, same-origin:
$ curl -s -b jar.txt --data-urlencode "csrf_token=<correct>" -d "to=mallory&amount=10" \
       http://localhost:5000/transfer -o /dev/null -w "%{http_code}\n"
302   (redirect to / -- success)

# The old GET attack no longer even matches a route:
$ curl -s -b jar.txt "http://localhost:5000/transfer?to=mallory&amount=300" -o /dev/null -w
"%{http_code}\n"
405   (Method Not Allowed)
```

In the browser, reopening `evil_post.html` or `evil_get.html` against the secure app now does nothing: the forged request either lacks a token entirely (blocked) or, because the attacker cannot read Alice's page, could never have obtained the correct token to put in it in the first place.

8. Testing for CSRF

8.1 Manual testing checklist

1. List every **state-changing** action in the application (anything that adds, changes or deletes data, or changes settings).
2. For each action, capture a legitimate request in an intercepting proxy (Burp Suite or OWASP ZAP, introduced in Unit 6).
3. Check whether the request relies only on cookies for authentication (no extra header or token the proxy had to add).
4. Remove or alter any token/parameter that looks like a CSRF token and resend the request. If it still succeeds, the application is vulnerable.
5. Build a minimal HTML proof-of-concept page (as in Section 4) reproducing the request and open it in a fresh browser session that is logged in to the target, to confirm real-world exploitability.
6. Check that **GET** requests never perform state changes.
7. Check cookie flags (`Secure`, `HttpOnly`, `SameSite`) in the response headers.
8. Re-test after any claimed fix, including with a token copied from a **different** session, to make sure the check is truly per-session.

8.2 Tool support

Tool	Relevant feature
Burp Suite / OWASP ZAP	"Generate CSRF PoC" tooling creates a ready-to-use attack HTML page from a captured request
Browser DevTools	Inspect `Set-Cookie` headers for `SameSite`, `HttpOnly`, `Secure`; inspect the Network tab for tokens
OWASP ZAP automated scan	Flags forms that lack an anti-CSRF token
Framework-specific checks	Many frameworks (Django, Rails, Spring, ASP.NET) include CSRF middleware that can be verified as enabled in configuration review

8.3 Automated regression tests

The supplied `test_csrf.py` (Appendix A, also used while preparing these notes) is a good model for a repeatable regression test: it asserts that the vulnerable app can be exploited (documenting the bug) and that the secure app rejects six different forged variations while still accepting a correctly formed request. Embedding a test like this in a CI pipeline stops the protection from being accidentally removed later.

Excerpt from test_csrf.py, confirmed passing during preparation of this unit

```
def test_secure():
    ...
    assert c.post("/transfer", data={...no token...}).status_code == 403
    assert c.post("/transfer", data={...wrong token...}).status_code == 403
    assert c.post("/transfer", data={...right token, foreign Origin...}).status_code == 403
    assert c.get('/transfer?...').status_code == 405        # GET no longer accepted
    r = c.post("/transfer", data={...correct token, same origin...})
    assert r.status_code == 302                                  # legitimate request still works
```

8.4 Framework defaults: what to check in real projects

In professional development, you will rarely hand-roll token logic as we did in Section 7; most frameworks include it. The teaching exercise builds understanding so that you can **recognise, configure and audit** these built-in protections rather than leave them disabled or misconfigured.

Framework	Built-in CSRF protection
Django (Python)	`django.middleware.csrf.CsrfViewMiddleware`, `{% csrf_token %}` template tag
Flask (Python)	Not built in to core Flask; use the `Flask-WTF` / `flask-seasurf` extension, or implement as in Section 7
Ruby on Rails	`protect_from_forgery` (enabled by default in generated applications)
ASP.NET Core	Antiforgery middleware and `[ValidateAntiForgeryToken]` / automatic tag helper support
Spring (Java)	Spring Security CSRF protection (enabled by default for state-changing methods)
Express (Node.js)	Not built in; historically `csurf` middleware (now deprecated — check current OWASP/Express guidance for a maintained alternative) or the double-submit pattern implemented directly

> **WARNING: Check it is actually switched on**
>
> A framework default is only a defence if nobody has disabled it. Penetration testers and code reviewers routinely find CSRF middleware explicitly turned off "to make testing easier" and never turned back on. Always verify the configuration, do not just assume the framework protects you.

9. Scenarios, Activities and Assessment

9.1 Scenario: the SACCO member portal

A savings and credit cooperative (SACCO) builds a member portal where members can request a loan top-up by submitting a form with a member ID and an amount. A tester notices the request is a simple GET with no token: `GET /loan/topup?member=4521&amount=20000`. A link embedded in a WhatsApp forward, disguised as a "loan calculator", causes any logged-in member who clicks it to submit a top-up request without realising it.

- **Which condition from Section 3 is violated?** All three: a relevant action exists, authentication is cookie-only, and nothing in the request is unpredictable.
- **Fix:** convert to POST, add a per-session anti-CSRF token, set `SameSite=Lax` on the session cookie, and require re-confirmation (OTP) for top-ups above a threshold.

9.2 Scenario: the admin panel with "clever" protection

A developer protects the admin panel by checking a custom header `X-Requested-With: XMLHttpRequest` on every POST, reasoning that "a plain HTML form cannot send custom headers, so this stops CSRF". This is **true for plain HTML forms**, but the developer has not considered that some older browser plugins, misconfigured CORS policies, or Flash-based techniques (historically) could add headers, and more importantly, has not added any secret value at all — only a header that is easy for script-capable attack techniques to replicate. Discuss: is this better than nothing? What is still missing?

Guidance for discussion: It raises the bar slightly against the very simplest attacks but provides **no cryptographic guarantee** and is not the OWASP-recommended primary control. A genuine per-session random token, verified server-side, is still required.

9.4 Discussion questions

1. Why does the same-origin policy not already prevent CSRF, given that it exists to protect data between origins?
2. Explain, in your own words, why `SameSite=Lax` stops the POST attack in Demo 2 but not the GET attack in Demo 1.
3. A colleague says, "We use HTTPS everywhere, so CSRF cannot happen." Explain why they are mistaken.
4. Why must the server compare tokens using a constant-time function such as `hmac.compare_digest` rather than `==`?
5. Explain why XSS can defeat a correctly implemented anti-CSRF token, and what that implies about the order in which a team should fix vulnerabilities.
6. Why is it important to protect the `/login` endpoint with CSRF defences too, not only actions taken after login?
7. Design a CSRF defence for a stateless mobile API that does not use cookies at all. Does it need the same protection? Why or why not?

9.5 Quick quiz (multiple choice)

Use as a 10-minute classroom quiz. The answer key is in Section 9.7.

Q1. CSRF primarily exploits which browser behaviour?

A. JavaScript execution
B. Automatic attachment of cookies to requests
C. DNS caching
D. Certificate validation

Q2. Which of these is NOT one of the three conditions needed for CSRF?

A. A relevant state-changing action exists
B. Authentication relies only on automatically supplied credentials
C. The victim must have a weak password
D. No unpredictable value is required in the request

Q3. Why can a CSRF attacker not simply read the victim's bank balance?

A. Banks do not use HTTP
B. The same-origin policy blocks the attacker's page from reading the response
C. CSRF tokens encrypt all responses
D. Balances are never shown on screen

Q4. Which SameSite value still allows a cookie on a top-level cross-site GET navigation?

A. None
B. Lax
C. Strict
D. Secure

Q5. The core anti-CSRF defence covered in this unit is called:

A. Session fixation
B. The synchroniser token pattern
C. Certificate pinning
D. Content Security Policy

Q6. An anti-CSRF token must be:

A. The same for every user, for simplicity
B. Predictable so support staff can verify it
C. Unpredictable, tied to the session, and checked on every state-changing request
D. Placed only in the URL query string

Q7. Why is changing a GET route to POST, alone, not enough to stop CSRF?

A. POST requests are not supported by HTML
B. Browsers can still auto-submit forged forms using POST
C. POST requests are always encrypted
D. POST requests cannot carry parameters

Q8. What does hmac.compare_digest protect against, compared to using ==?

A. SQL injection
B. A timing attack that could leak the token
C. Cross-site scripting
D. Denial of service

Q9. If an application is vulnerable to XSS, its anti-CSRF tokens:

A. Become stronger automatically
B. Can potentially be read and reused by the attacker's script, defeating the protection
C. Are irrelevant because XSS always uses POST
D. Automatically rotate every second

Q10. Which header can the server check, as an additional layer, to see if a request came from its own site?

A. Accept-Language
B. Origin
C. User-Agent
D. Content-Length

Q11. Login CSRF is an attack where:

A. The attacker steals the victim's password
B. The attacker forces the victim's browser to log in to the attacker's own account
C. The attacker performs a brute-force login
D. The attacker intercepts TLS traffic

Q12. Which of these is the WEAKEST standalone defence against CSRF?

A. A per-session random token checked server-side
B. Checking only that a session cookie is present
C. SameSite=Strict cookies
D. Re-authentication for sensitive actions

Q13. In the MiniBank demo, why was 127.0.0.1:8000 used for the attacker page instead of localhost:8000?

A. It is faster
B. To ensure the browser treats it as a different SITE from localhost:5000, unlike two localhost ports
C. It uses a different protocol
D. It is required by Flask

Q14. A secure implementation should respond to a missing or invalid CSRF token by:

A. Logging the user out silently with no error
B. Allowing the request once, then blocking the next one
C. Rejecting the request immediately (fail closed)
D. Redirecting to the home page without any status change

Q15. Which statement best describes the relationship between CSRF and SSRF?

A. They are the same attack with different names
B. In CSRF the victim's browser is tricked; in SSRF the server itself is tricked into making a request
C. SSRF only affects mobile apps
D. CSRF requires server-side code execution

9.6 Short-answer and applied questions

1. Define CSRF in your own words and explain the office-badge analogy. (3 marks)
2. List and explain the three conditions required for a CSRF attack to succeed. (6 marks)
3. Explain the difference between "origin" and "site", and why it matters for SameSite cookies. (4 marks)
4. Describe, step by step, how the synchroniser token pattern prevents CSRF. (6 marks)
5. A developer protects a form by storing the CSRF token in a cookie named `csrf` and comparing it to a hidden form field with the same value, but never ties the token to the server-side session. Identify the weakness and propose a fix. (5 marks)
6. Explain why `SameSite=Lax` alone would not have stopped the GET-based demo in Section 4.2. (4 marks)
7. Your team has both a reflected XSS vulnerability and a missing CSRF token on the same page. Which should be fixed first, and why? (4 marks)
8. Design a short CSRF test plan (five steps) that a diploma-level tester could follow against an unfamiliar web application. (5 marks)

10. Summary, Glossary and Quick Reference

10.1 Key takeaways

- **CSRF** tricks a victim's browser into sending a genuine, cookie-authenticated request to a site without the victim's intent. The attacker never needs the password or the session cookie value.
- The same-origin policy stops the attacker **reading** responses, but does **not** stop a page from **sending** many kinds of cross-site requests.
- Three conditions make CSRF possible: a relevant action, authentication based only on automatically supplied credentials, and no unpredictable value in the request.
- GET-based CSRF can succeed via a simple link or navigation even under `SameSite=Lax`; only `Strict` (or removing state changes from GET entirely) closes that gap.
- The primary defence is the **synchroniser (anti-CSRF) token pattern**: a random, session-bound token checked on every state-changing request, compared in constant time, failing closed on any mismatch.
- Supporting layers: explicit `SameSite` cookies, `Origin`/`Referer` (or Fetch Metadata) checks, and re-authentication for sensitive actions. Use several together.
- CSRF protection is undermined by **XSS**, so output encoding and input validation (Unit 6) remain essential.
- Test for CSRF by removing or tampering with tokens and resending requests, and automate regression tests so protection is never silently removed.

10.2 Glossary

Term	Definition
Anti-CSRF token	A random, unpredictable value issued by the server and required on state-changing requests
CSRF (XSRF)	Cross-Site Request Forgery; forcing a victim's browser to send an unwanted authenticated request
Double-submit cookie	CSRF defence comparing a token sent as a cookie against one sent as form data or a header
Fail closed	Denying the action when a security check cannot be completed or fails
Fetch Metadata (Sec-Fetch-Site)	Browser-supplied header describing the relationship between the requesting page and the target
HttpOnly	Cookie attribute preventing JavaScript from reading the cookie
Login CSRF	Forcing a victim to log in to the attacker's account rather than their own
Origin	Scheme + host + port combination identifying where a request or script comes from
Same-origin policy (SOP)	Browser rule restricting scripts from reading responses from a different origin
Same-site	Relationship between two URLs based on the registrable domain, ignoring scheme/port
SameSite (cookie attribute)	Controls whether a cookie is sent with cross-site requests (None, Lax, Strict)
Synchroniser token pattern	The standard anti-CSRF design: server-issued, session-bound, verified token

Term	Definition
XSS	Cross-site scripting; injecting script that runs in the target's own origin, which can defeat CSRF tokens

10.3 One-page cheat sheet



10.4 Self-study and further reading

- **OWASP Cross-Site Request Forgery Prevention Cheat Sheet** (cheatsheetseries.owasp.org) — the authoritative, regularly updated practical guide.
- **OWASP Web Security Testing Guide (WSTG)**, CSRF testing chapter.
- **MDN Web Docs**: “SameSite cookies”, “Same-origin policy”, “Fetch Metadata Request Headers”.
- **PortSwigger Web Security Academy**: free CSRF labs with step-by-step explanations.
- **Practice platforms (lab only):** OWASP Juice Shop and DVWA (also used in Unit 6) both contain CSRF-related challenges.
- Always check **current** browser release notes for SameSite defaults before teaching or deploying, since browser vendors continue to change these defaults.

KEY POINT

CSRF is a reminder that security is not only about keeping secrets secret. A session cookie can be exactly where it should be, on exactly the right device, and an attack can still succeed, because the application never asked “did the user really mean to do this?” Build applications that always ask.

Appendix A: Full Lab Code Listings

These files were written and verified while preparing this unit (all tests passed; see Section 7.6 for captured output). Copy each listing into a file of the same name in one folder.

REMIND ME TO SHARE WITH YOU THE GITHUB LINK FOR THE SOURCE CODE FILES

Verified result when run: `ALL TESTS PASSED` (captured output reproduced in Section 7.6 and during preparation of this unit).